

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÀI TẬP LỚN MÔN HỌC
MÔ HÌNH HÓA TOÁN HỌC

ĐỀ TÀI:
Symbolic and Algebraic Reasoning in Petri Nets

Nhóm: 33 - Học kỳ: 251
GVHD: TS. Trịnh Văn Giang

Sinh viên thực hiện:

STT	MSSV	Họ và tên	Lớp
1	2310123	Nguyễn Thị Kim Anh	TN02
2	2410355	Nguyễn Thanh Bình	L03
3	2410709	Nguyễn Thành Đạt	L03
4	2312318	Phạm Thảo Ngọc	TN02
5	2413477	Lương Hoàng Vĩnh Tiến	TN02

TP. Hồ Chí Minh, tháng 11 năm 2025

Mục lục

Phân công công việc	3
1 Giới thiệu	4
2 Quy trình xử lý (Workflow)	4
3 Task 1: Phân tích và đọc file PNML	5
3.1 Cơ sở lý thuyết	5
3.1.1 Cấu trúc mạng petri	5
3.1.2 Marking	5
3.2 Thiết kế và cài đặt	6
3.2.1 Cấu trúc tổng quát của 1 file PNML	6
3.2.2 Cấu trúc dữ liệu được sử dụng để lưu mạng Petri	7
3.2.3 Quy trình parse file PNML	7
3.2.4 Quy trình parse file PNML	7
4 Task 2: Tính toán Reachability bằng phương pháp tường minh	9
4.1 Cơ sở lý thuyết	9
4.1.1 Transition được kích hoạt	9
4.1.2 Firing transition	9
4.1.3 Bài toán Reachability	9
4.2 Thiết kế và cài đặt	10
4.2.1 Kiểm tra transition được kích hoạt	10
4.2.2 Hàm fire	10
4.3 Giải thuật BFS để tính Reachability	10
4.4 Nhận xét	11
5 Task 3: Tính toán Reachability bằng BDD	11
5.1 Cơ sở lý thuyết	11
5.1.1 Biểu diễn marking bằng Boolean function	11
5.1.2 Transition relation $T(s, s')$	11
5.1.3 Post operator và fixed-point iteration	12
5.2 Thiết kế và cài đặt	12
5.2.1 Cấu trúc BDD	12
5.2.2 Transition relation tối giản	13
5.2.3 Thuật toán symbolic reachability (fixed-point)	13
5.2.4 Flowchart thuật toán	14
5.3 Kết quả thực nghiệm	14

5.3.1	Explicit Reachability (Task 2)	15
5.3.2	Symbolic Reachability bằng BDD (Task 3)	15
5.4	So sánh explicit và symbolic	16
5.5	Nhận xét	16
5.6	Khó khăn và hướng cải tiến	16
6	Task 4: Phát hiện Deadlock bằng ILP kết hợp BDD	16
6.1	Cơ sở lý thuyết	17
6.2	Mô hình hóa ILP	17
6.3	Thiết kế và cài đặt	18
6.4	Kết quả thực nghiệm	18
6.5	Nhận xét	19
7	Task 5: Tối ưu hóa trên tập Reachable Markings	20
7.1	Cơ sở lý thuyết	20
7.2	Thiết kế và cài đặt	20
7.2.1	Nguyên tắc hoạt động:	20
7.2.2	Cài đặt Hàm Tối ưu Hóa (optimize)	21
7.2.3	Phân tích Cài đặt	21
7.3	Kết quả thực nghiệm	22
7.3.1	Explicit Reachability (Task 2)	22
7.3.2	Kết quả optimization (Task 5)	23
7.4	Nhận xét thuật toán	23
7.4.1	Đánh giá Phương pháp Liệt kê Tường minh (Explicit Enumeration)	23
7.4.2	Hạn chế và Thách thức	23
7.5	Cách cải tiến	24
8	Tài liệu tham khảo	25



Phân công công việc

STT	MSSV	Họ và tên	Nhiệm vụ	Hoàn thành
1	2310123	Nguyễn Thị Kim Anh	Task 3	100%
2	2410355	Nguyễn Thanh Bình	Task 5	100%
3	2410709	Nguyễn Thành Đạt	Task 4	100%
4	2312318	Phạm Thảo Ngọc	Task 3	100%
5	2413477	Lương Hoàng Vĩnh Tiến	Task 1 + 2	100%

Bài tập lớn được triển khai và quản lý trên GitHub tại: [MHH-Petri-Nets](#)

1 Giới thiệu

Petri Net là mô hình toán học dùng để mô tả hành vi của các hệ thống song song và dựa trên sự kiện, thông qua ba thành phần: *place*, *transition* và *token*. Nhờ cấu trúc trực quan và mang tính hình thức, Petri Net được sử dụng rộng rãi trong mô phỏng và kiểm chứng các hệ thống thực.

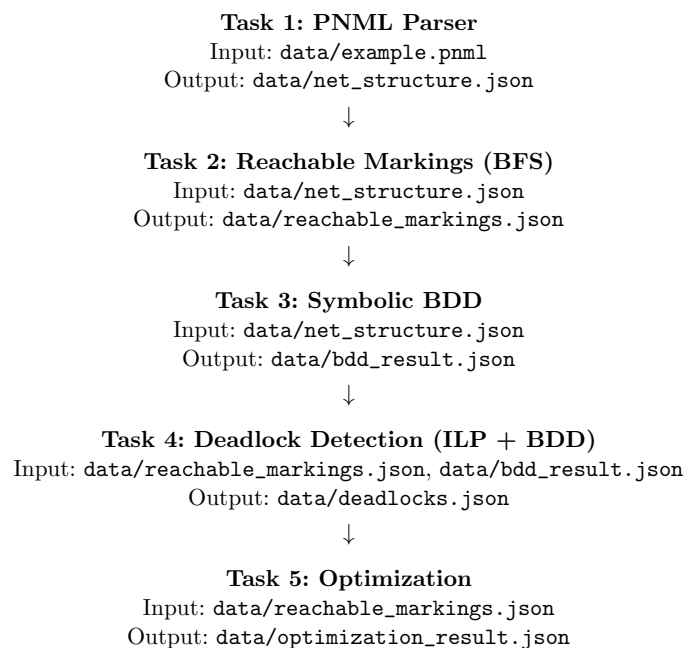
Trong báo cáo này, nhóm xây dựng một pipeline gồm năm nhiệm vụ:

- **Task 1:** Phân tích tệp PNML và sinh cấu trúc mạng.
- **Task 2:** Tính reachable markings bằng BFS/DFS.
- **Task 3:** Phân tích reachability bằng BDD.
- **Task 4:** Phát hiện deadlock dùng BDD kết hợp ILP.
- **Task 5:** Tối ưu hóa tuyến tính trên tập reachable.

Báo cáo trình bày thiết kế cài đặt, kết quả thực nghiệm và so sánh giữa hai phương pháp explicit và symbolic, nhằm xây dựng một công cụ phân tích Petri Net hiệu quả và có khả năng mở rộng.

2 Quy trình xử lý (Workflow)

Chương trình được tổ chức thành 5 mô-đun tương ứng với 5 nhiệm vụ của bài tập lớn. Các mô-đun trao đổi dữ liệu thông qua các tệp JSON đặt trong thư mục `data/`. Toàn bộ pipeline được mô tả như sau:



Mô tả tổng quát các bước

- **Task 1 – PNML Parser:** Đọc tệp `example.pnml` và sinh ra `net_structure.json` chứa places, transitions, arcs và marking ban đầu.
- **Task 2 – Reachable Markings:** Sử dụng BFS để liệt kê toàn bộ các marking có thể đạt được và lưu vào `reachable_markings.json`.
- **Task 3 – Symbolic BDD:** Xây dựng biểu diễn symbolic của tập reachable markings bằng *Reduced Ordered Binary Decision Diagrams* (ROBDDs) từ `net_structure.json`. Kết quả và dữ liệu phục vụ phát hiện deadlock được lưu trong `bdd_result.json`.
- **Task 4 – Deadlock Detection:** Kết hợp BDD và tập reachable để phát hiện các trạng thái deadlock, kết quả được lưu trong `deadlocks.json`.
- **Task 5 – Optimization:** Dựa trên vector trọng số c (từ `CArr.json`), chương trình xác định marking tối ưu trong tập reachable và lưu vào `optimization_result.json`.

3 Task 1: Phân tích và đọc file PNML

3.1 Cơ sở lý thuyết

Các thành phần của một mạng Petri:

Một mạng Petri gồm ba thành phần chính:

Place - biểu diễn bằng hình tròn/ellipse, đóng vai trò *thụ động*, lưu trữ token. Trạng thái của place mang tính rời rạc vì token là số nguyên không âm.

Transition - biểu diễn bằng hình vuông/chữ nhật, là *thành phần chủ động* thực hiện tiêu thụ và sinh token qua quá trình firing.

Arc - liên kết có hướng giữa place và transition (hoặc ngược lại), mô tả quan hệ logic hoặc luồng điều khiển trong hệ thống, không phải một thực thể vật lý.

3.1.1 Cấu trúc mạng petri

Ta ký hiệu tập các place, transition và arc lần lượt là P , T , và F , và xem các arc là các cặp, tức là xem F như một quan hệ $F \subseteq (P \times T) \cup (T \times P)$. Từ đó ta có

$$N = (P, T, F)$$

là một *cấu trúc mạng*. Các place và transition là các phần tử của N , F gọi là *quan hệ dòng chảy* của N .

3.1.2 Marking

Marking μ là một hàm gán số lượng token cho từng place:

$$\mu : P \rightarrow \mathbb{N}.$$

Marking cũng có thể biểu diễn dưới dạng vector:

$$\mu = (\mu_1, \mu_2, \dots, \mu_{|P|}), \quad \mu_i \in \mathbb{N}.$$

Một mạng Petri được gọi là **1-safe** nếu mọi place đều chứa không quá một token tại mọi thời điểm, tức là $\mu_i \in \{0, 1\}$ với mọi i .

3.2 Thiết kế và cài đặt

3.2.1 Cấu trúc tổng quát của 1 file PNML

PNML là viết tắt của Petri Net Markup Language, là chuẩn XML để mô tả mạng Petri, có cấu trúc 3 tầng: `<pnml>` $\rightarrow$ `<net>` $\rightarrow$ `<page>` giúp trao đổi mô hình, đơn giản hóa việc parsing. Trong phạm vi bài tập lớn này, file PNML được sử dụng làm nguồn dữ liệu đầu vào để lấy danh sách place, transition, các cung (arc), cùng với marking ban đầu của mạng. Trong 1 file PNML, các thành phần của mạng petri xuất hiện dưới dạng các tag:

Thành phần	Tag PNML	Thuộc tính quan trọng
Place	<code><place></code>	id, <code><initialMarking></code>
Transition	<code><transition></code>	id
Arc	<code><arc></code>	id, source, target

File PNML được tổ chức dưới dạng cấu trúc 3 tầng như sau:

```

1 <pnml>
2   <net id="...">
3     <page id="...">
4
5       <!-- Tags can appear in any order -->
6       <place id="..."> ... </place>
7       <transition id="..."> ... </transition>
8       <arc id="..." source="..." target="..." />
9
10      <!-- Extra tags -->
11      <name> ... </name>
12      <graphics> ... </graphics>
13      <toolspecific> ... </toolspecific>
14
15    </page>
16  </net>
17 </pnml>

```

PNML không yêu cầu các phần tử `<place>`, `<transition>` và `<arc>` phải được nhóm lại hay xuất hiện theo bất kỳ thứ tự cố định nào; chúng có thể phân bố tự do và xen kẽ nhau bên trong `<page>`. Vì vậy, trình phân tích (parser) được xây dựng theo hướng duyệt toàn bộ cây XML và xử lý phần tử dựa trên tên thẻ thay vì dựa vào vị trí của chúng trong tài liệu.

3.2.2 Cấu trúc dữ liệu được sử dụng để lưu mạng Petri

Sau khi phân tích file PNML, mạng Petri được lưu trữ trong lớp `PetriNet` thông qua các cấu trúc dữ liệu sau:

- **places:** `dict[str, int]` : Ánh xạ mỗi id của place sang số lượng token ban đầu (marking). Cho phép truy cập và cập nhật marking theo thời gian $O(1)$.
- **transitions:** `dict[str, bool]` : Lưu danh sách các transition của mạng dưới dạng một từ điển, trong đó khóa là id của transition và giá trị là `True`.
- **arcs:** `list[[str, str]]` : Danh sách các cung của mạng, mỗi phần tử là một cặp `[source, target]`. Danh sách được dùng thay vì từ điển vì thứ tự đọc không quan trọng và không cần tra cứu trực tiếp theo khóa.
- **initial_marking:** `dict[str, int]` : Dữ liệu đánh dấu ban đầu của mạng được giữ nguyên từ `places`, và được xuất dưới trường `"initial_marking"` trong tệp JSON đầu ra.

Cấu trúc tổng thể của đối tượng JSON được tạo bởi hàm `to_dict()` có dạng:

```
{
  "places": [string],
  "transitions": [string],
  "arcs": [[string, string]],
  "initial_marking": { string: int }
}
```

3.2.3 Quy trình parse file PNML

3.2.4 Quy trình parse file PNML

Quá trình phân tích tệp PNML được thực hiện qua các bước:

- **Tải và kiểm tra tệp:** sử dụng `xml.etree.ElementTree` để đọc PNML; nếu tệp không tồn tại hoặc XML sai định dạng, sinh `PNMLFormatError`.
- **Duyệt cây XML:** parser duyệt toàn bộ cây bằng `root.iter()` để phát hiện các phần tử `place`, `transition` và `arc` (không phụ thuộc thứ tự xuất hiện).
- **Xử lý place:** thu thập id và marking ban đầu (nếu có); mặc định `marking = 0`.
- **Xử lý transition:** ghi nhận id của mỗi `<transition>` vào danh sách `transitions`.
- **Xử lý arc:** lấy `source-target` và thêm vào danh sách cung.
- **Kiểm tra tính hợp lệ:** đảm bảo có ít nhất một place, và mỗi cung phải có `source/target` hợp lệ, không tồn tại cung kiểu `place→place` hoặc `transition→transition`.

Parser cần thực hiện một số kiểm tra cơ bản để tránh lỗi:

Loại lỗi	Mô tả
Thiếu tệp PNML	Tệp đầu vào không tồn tại hoặc đường dẫn sai, dẫn đến không thể đọc dữ liệu.
Không phân tích được XML	Cấu trúc XML không hợp lệ, gây lỗi khi sử dụng <code>ElementTree.parse()</code> .
Thiếu thuộc tính <code>id</code> ở <code>place</code> hoặc <code>transition</code>	Một phần tử <code><place></code> hoặc <code><transition></code> không có thuộc tính <code>id</code> .
Thiếu <code>source</code> hoặc <code>target</code> trong <code>arc</code>	Một phần tử <code><arc></code> bị thiếu một trong hai thuộc tính quan trọng <code>source</code> hoặc <code>target</code> .
Tham chiếu đến nút không tồn tại	<code>source</code> hoặc <code>target</code> của <code>arc</code> không nằm trong tập <code>place</code> hoặc <code>transition</code> .
Cung không hợp lệ: <code>place → place</code>	Parser không cho phép cung nối từ <code>place</code> sang <code>place</code> . Đây không phải cấu trúc hợp lệ trong Petri net.
Cung không hợp lệ: <code>transition → transition</code>	Parser không cho phép cung nối từ <code>transition</code> sang <code>transition</code> .
Giá trị initial marking không hợp lệ	Marking không phải số nguyên hoặc không thể chuyển đổi sang kiểu <code>int</code> .

Sau khi phân tích file PNML, mô hình mạng Petri được chuyển về dạng JSON với các trường:

- "places": danh sách id của các place.
- "transitions": danh sách id của transition.
- "arcs": danh sách cặp [source, target].
- "initial_marking": ánh xạ place → số token ban đầu.

Ví dụ: Cho file PNML như sau:

```

1 <pnml>
2   <net id="net1">
3     <page id="p1">
4       <transition id="t1"/>
5       <place id="p1"/>
6       <arc id="a1" source="p1" target="t1"/>
7       <transition id="t2"/>
8       <arc id="a2" source="t1" target="p2"/>
9       <place id="p2"/>
10      <arc id="a3" source="p2" target="t2"/>
11    </page>
12  </net>
13 </pnml>

```

Sau khi chạy qua parser sẽ có được cấu trúc mạng như sau:

```
1 {  
2   "places": ["p1", "p2"],  
3   "transitions": ["t1", "t2"],  
4   "arcs": [  
5     ["p1", "t1"],  
6     ["t1", "p2"],  
7     ["p2", "t2"]  
8   ],  
9   "initial_marking": {"p1": 0, "p2": 0}  
10 }
```

4 Task 2: Tính toán Reachability bằng phương pháp tường minh

4.1 Cơ sở lý thuyết

4.1.1 Transition được kích hoạt

Trong mạng Petri 1-safe, một transition t được xem là *được kích hoạt* tại marking μ nếu:

$$\forall p \in Pre(t), \mu(p) = 1.$$

Transition không có input (tức $Pre(t) = \emptyset$) được xem là không hợp lệ và không được kích hoạt.

4.1.2 Firing transition

Khi một transition được kích hoạt và được fire, marking mới μ' thu được bằng cách:

$$\mu'(p) = \begin{cases} \mu(p) - 1, & p \in Pre(t), \\ \mu(p) + 1, & p \in Post(t), \\ \mu(p), & \text{ngược lại.} \end{cases}$$

Trong mạng 1-safe, mọi giá trị luôn thuộc miền $\{0, 1\}$ sau khi firing.

4.1.3 Bài toán Reachability

Từ marking ban đầu M_0 , mục tiêu là liệt kê toàn bộ marking có thể đạt được thông qua các chuỗi firing:

$$Reach(N, M_0) = \{ M \mid M_0 \xrightarrow{*} M \}.$$

4.2 Thiết kế và cài đặt

4.2.1 Kiểm tra transition được kích hoạt

```
function is_enabled(t, marking):
    input_places = get_inputs(t)
    if input_places is empty: return False
    for p in input_places:
        if marking[p] == 0: return False
    return True
```

4.2.2 Hàm fire

```
function fire(t, marking):
    new = copy(marking)
    inputs = get_inputs(t)
    outputs = get_outputs(t)
    for p in inputs: new[p] -= 1
    for p in outputs: new[p] += 1
    return new
```

4.3 Giải thuật BFS để tính Reachability

Reachability được tính bằng Breadth-First Search (BFS), trong đó:

- mỗi node tương ứng với một marking,
- tồn tại cạnh $M \rightarrow M'$ nếu có transition hợp lệ tạo ra M' ,
- tập visited dùng để tránh duyệt lại,
- queue tổ chức thứ tự duyệt theo chiều rộng.

```
function BFS-Reachability(M0):
    queue = [M0]
    visited =
    reachable =
    while queue not empty:
        M = queue.pop()
        if M in visited: continue
        visited.add(M)
        reachable.add(M)
        for each transition t:
            if is_enabled(t, M):
                M' = fire(t, M)
                if M' not in visited: queue.push(M')
    return reachable
```

4.4 Nhận xét

- **Ưu điểm:** Phương pháp explicit đơn giản, dễ hiểu và dễ kiểm chứng; phù hợp cho mô hình nhỏ.
- **Nhược điểm:** Số marking có thể tăng theo cấp số nhân, dẫn đến hiện tượng *state-space explosion*. Chi phí bộ nhớ tăng nhanh do phải lưu toàn bộ tập *visited*.

5 Task 3: Tính toán Reachability bằng BDD

Task 3 giữ vai trò trung tâm trong pipeline, hiện thực **symbolic reachability analysis** dựa trên *Binary Decision Diagrams* (BDD). Khác với phương pháp explicit ở Task 2, cách tiếp cận symbolic không liệt kê từng trạng thái riêng lẻ mà biểu diễn toàn bộ tập reachable dưới dạng một hàm Boolean được nén bằng *Reduced Ordered BDD* (ROBDD). Nhờ khai thác cấu trúc và tính lặp lại của không gian trạng thái, phương pháp này cho phép xử lý các mô hình lớn hiệu quả hơn.

Kết quả của Task 3 là một ROBDD biểu diễn tập reachable, được lưu trong `bdd_result.json`. BDD này cung cấp đầy đủ thông tin để phục vụ các phân tích tiếp theo trong Task 4, bao gồm kiểm tra deadlock và xây dựng bài toán tuyến tính (ILP).

5.1 Cơ sở lý thuyết

5.1.1 Biểu diễn marking bằng Boolean function

Trong mạng Petri 1-safe, mỗi place p_i có giá trị token $m_i \in \{0, 1\}$. Do đó, một marking $M = (m_1, \dots, m_{|P|})$ có thể được biểu diễn dưới dạng một *Boolean cube*:

$$cube(M) = \bigwedge_{i=1}^{|P|} \begin{cases} p_i, & \text{khi } m_i = 1, \\ \neg p_i, & \text{khi } m_i = 0. \end{cases}$$

Tập reachable markings R tương ứng với Boolean function:

$$F_R = \bigvee_{M \in R} cube(M),$$

được rút gọn bằng ROBDD. ROBDD nén hàm Boolean bằng cách loại bỏ nhánh trùng lặp và đảm bảo mỗi node có trật tự biến cố định. Kích thước của ROBDD phụ thuộc mạnh vào *variable ordering*; các thứ tự xen kẽ (interleaving ordering) thường cho kết quả nhỏ hơn đối với các bài toán reachability hai chiều s và s' .

5.1.2 Transition relation $T(s, s')$

Trong symbolic model checking, hành vi của mạng Petri được mã hoá bằng *transition relation* $T(s, s')$, một công thức Boolean mô tả mọi cặp trạng thái (s, s') sao cho có thể kích hoạt một transition từ s để thu được s' .

Đối với mỗi transition t , ta có công thức:

$$T_t(s, s') = \left(\bigwedge_{p \in \text{Pre}(t)} p \right) \wedge \left(\bigwedge_{p' \in \text{Post}(t)} p' \right) \wedge \left(\bigwedge_{p \in \text{Pre}(t)} \neg p' \right),$$

trong đó các biến p' biểu diễn trạng thái kế tiếp. Transition relation tổng thể là phép hợp của tất cả T_t :

$$T = \bigvee_{t \in T} T_t.$$

5.1.3 Post operator và fixed-point iteration

Cho một tập trạng thái symbolic $R(s)$, toán tử *Post operator* được định nghĩa:

$$\text{Post}(R) = \exists s. (R(s) \wedge T(s, s')),$$

trong đó *existential abstraction* loại bỏ các biến trạng thái hiện tại s , chỉ giữ lại biến s' . Để đưa kết quả trở lại không gian trạng thái ban đầu, phép *variable renaming* được áp dụng:

$$p' \mapsto p.$$

Phân tích reachability được thực hiện bằng *fixed-point iteration*:

$$R_{k+1} = R_k \cup \text{Post}(R_k), \quad R_0 = \text{cube}(M_0),$$

và thuật toán dừng khi đạt fixed point:

$$R_{k+1} = R_k.$$

Trong symbolic verification, đây là kỹ thuật chuẩn để xác định toàn bộ reachable state-space mà không cần liệt kê tường minh từng trạng thái.

5.2 Thiết kế và cài đặt

5.2.1 Cấu trúc BDD

Hệ thống sử dụng một bộ máy BDD tự cài đặt, tuân thủ đầy đủ quy tắc của *Reduced Ordered Binary Decision Diagram* (ROBDD). Mỗi node trong BDD có dạng bộ ba:

$$(var_index, low, high),$$

trong đó *low* và *high* lần lượt là con trỏ tới nhánh 0 và nhánh 1. Để đảm bảo tính rút gọn:

- **Unique-table** được sử dụng để bảo đảm rằng không có hai node trùng lặp, từ đó duy trì invariant của ROBDD.

- **Apply-cache** ghi nhớ kết quả của các phép toán AND/OR để tránh tính toán lặp lại, giúp tăng tốc đáng kể trong quá trình fixed-point.
- **Variable ordering**: hệ thống áp dụng chiến lược **interleaving**:

$$[p_1, p'_1, p_2, p'_2, \dots, p_n, p'_n],$$

thay vì gom toàn bộ biến hiện tại và biến tương lai thành hai khối tách biệt. Thứ tự này thường cho BDD nhỏ hơn đáng kể khi thực hiện symbolic reachability.

Ngoài ra, hệ thống sử dụng thêm một mô-đun *symbolic model counting* để tính số marking reachable trực tiếp từ ROBDD mà không cần liệt kê.

5.2.2 Transition relation tối giản

Dựa trên cấu trúc mạng Petri, hệ thống xây dựng quan hệ chuyển tiếp $T(s, s')$ bằng dạng mã hoá tối giản. Với mỗi transition t :

$$T_t(s, s') = \left(\bigwedge_{p \in \text{Pre}(t)} p \right) \wedge \left(\bigwedge_{p \in \text{Post}(t)} p' \right) \wedge \left(\bigwedge_{p \in \text{Pre}(t)} \neg p' \right).$$

Dạng biểu diễn này chỉ ràng buộc những biến thực sự thay đổi khi firing, giúp giảm đáng kể kích thước ROBDD so với các biến thể đầy đủ đòi hỏi mã hoá $p' = p$ cho tất cả place không liên quan.

Transition relation tổng thể:

$$T = \bigvee_{t \in T} T_t.$$

5.2.3 Thuật toán symbolic reachability (fixed-point)

Hệ thống thực hiện phân tích reachable markings hoàn toàn bằng symbolic BDD, không sử dụng bất kỳ phương pháp explicit nào. Thuật toán tổng quát:

```
Initialize R = cube(M0)

Repeat:
  RT = R AND T
  Post_next = P . RT
  Post = rename(p' → p)
  R_new = R OR Post
  If R_new == R: break
  R = R_new
End
```

Ý nghĩa các bước:

- **AND**: tìm các cặp (s, s') xuất phát từ R và hợp lệ theo T .
- **Existential abstraction**: loại bỏ các biến trạng thái hiện tại s , giữ lại s' .

- **Renaming:** chiếu kết quả về không gian trạng thái gốc bằng phép đổi $p' \mapsto p$.
- **Fixed-point:** dừng lại khi không còn trạng thái mới xuất hiện.

Sau khi đạt fixed-point, hệ thống áp dụng symbolic counting để tính:

$$|R| = \text{SATCOUNT}(F_R).$$

5.2.4 Flowchart thuật toán

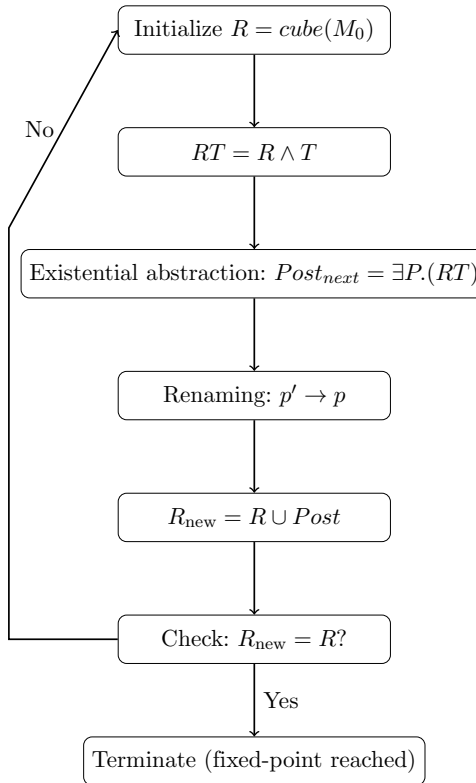


Figure 1: Symbolic reachability computation using ROBDD

5.3 Kết quả thực nghiệm

Nhóm tiến hành thử nghiệm Task 2 và Task 3 trên mạng Petri gồm 4 place và 4 transition dạng *split-join*, với marking ban đầu:

$$M_0 = (1, 0, 0, 0).$$

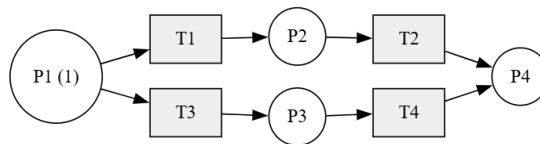


Figure 2: Mạng Petri sử dụng trong thực nghiệm

5.3.1 Explicit Reachability (Task 2)

Phương pháp BFS liệt kê trực tiếp toàn bộ không gian trạng thái và thu được đúng 4 marking khả đạt:

$$(1, 0, 0, 0), (0, 1, 0, 0), (0, 0, 1, 0), (0, 0, 0, 1).$$

Kết quả thực nghiệm:

```
"num_markings": 4,  
"build_time_seconds": 0.000239,  
"memory_usage_mb": 0.000122
```

Explicit hoạt động nhanh và tiêu thụ bộ nhớ rất thấp nhờ số lượng trạng thái nhỏ.

5.3.2 Symbolic Reachability bằng BDD (Task 3)

Trong chế độ symbolic, quan hệ chuyển tiếp $T(s, s')$ được mã hoá trên 8 biến:

$$(P1, P2, P3, P4, P1', P2', P3', P4'),$$

và quá trình lặp fixed-point được áp dụng:

$$R_{k+1} = R_k \cup \text{Post}(R_k), \quad R_0 = \text{cube}(M_0).$$

Kết quả thực nghiệm:

```
Reachable markings: 4  
BDD nodes: 142  
Iterations: 3  
Build time (sec): 0.000998  
Memory usage: 0.0134 MB
```

Phân tích kết quả

- **142 node ROBDD:** kích thước phản ánh độ phức tạp của hàm chuyển tiếp $T(s, s')$, không phải số lượng marking (chỉ 4). ROBDD phải mô tả hàm Boolean trên 8 biến.
- **Ba vòng fixed-point:** tương ứng ba bước lan truyền token trong cấu trúc pipeline

$$P1 \rightarrow (P2 \text{ hoặc } P3) \rightarrow P4.$$

- **Thời gian chạy ≈ 1 ms:** cao hơn explicit do chi phí tạo và rút gọn ROBDD, nhưng vẫn rất nhỏ đối với mô hình này.
- **Bộ nhớ ≈ 0.0134 MB:** lớn hơn explicit nhưng vẫn ở mức rất thấp, cho thấy symbolic có khả năng mở rộng tốt hơn khi state-space lớn.

5.4 So sánh explicit và symbolic

Tiêu chí	Explicit (BFS)	Symbolic (ROBDD)
Biểu diễn	Danh sách marking	Hàm $R(s)$ trên BDD
Cách mở rộng	Duyệt từng trạng thái	Phép toán logic: AND, OR, EXISTS
Bộ nhớ	Tăng theo số marking	Phụ thuộc ordering; nén mạnh với mô hình lớn
Thời gian	Nhanh khi nhỏ; nổ trạng thái khi lớn	Phụ thuộc $T(s, s')$ hơn là số marking
Khả năng mở rộng	Thấp	Rất cao; phù hợp verification
Nhược điểm	Không xử lý được state-space lớn	Nhạy với variable ordering

Table 1: So sánh tổng quan explicit vs. symbolic

5.5 Nhận xét

Explicit phù hợp mô hình nhỏ và dễ quan sát, nhưng không mở rộng tốt khi số marking tăng nhanh. Symbolic reachability sử dụng ROBDD để xử lý tập trạng thái dưới dạng hàm Boolean, cho khả năng mở rộng vượt trội và chi phí bộ nhớ ổn định, dù kích thước ROBDD không phản ánh trực tiếp số marking.

5.6 Khó khăn và hướng cải tiến

- **Variable ordering:** ROBDD rất nhạy với thứ tự biến; có thể giảm 30–70% node nếu dùng *dynamic reordering* (như sifting).
- **Existential abstraction tốn chi phí:** có thể phát sinh BDD trung gian lớn; kỹ thuật *early quantification* giúp giảm đáng kể.
- **Transition relation lớn:** có thể chia nhỏ T thành nhiều phần (*partitioned transition relation*) để giảm kích thước BDD.
- **Heuristic ordering theo cấu trúc mạng:** sắp xếp biến theo mức độ kết nối hoặc theo thứ tự firing giúp giảm node.
- **Tối ưu symbolic counting:** có thể cải thiện bằng memoisation sâu hơn hoặc tách sub-BDD.

Các kỹ thuật này được sử dụng trong các thư viện công nghiệp như CUDD và VIS, và có thể giúp symbolic reachability mở rộng tốt hơn cho các mô hình lớn.

6 Task 4: Phát hiện Deadlock bằng ILP kết hợp BDD

Nhiệm vụ này kết hợp sức mạnh của Quy hoạch tuyến tính nguyên (ILP) và Biểu đồ quyết định nhị phân (BDD) để giải quyết bài toán phát hiện Deadlock. Trong khi BDD cung cấp thông tin chính xác tuyệt đối về tính đạt được (Reachability), ILP cung cấp một cơ chế tìm kiếm heuristic hiệu quả để xác định các trạng thái "chết" (Dead markings) tiềm năng mà không cần duyệt toàn bộ không gian trạng thái.

6.1 Cơ sở lý thuyết

Một marking M trong mạng Petri được gọi là **Dead Marking** nếu tại trạng thái đó, không có bất kỳ transition nào có thể kích hoạt (enabled).

Trong ngữ cảnh mạng Petri 1-safe (nơi số token tại mỗi place p chỉ có thể là 0 hoặc 1), điều kiện để một transition t *không thể kích hoạt* là tổng số token tại các input places không đủ để thỏa mãn điều kiện bắn:

$$\sum_{p \in \bullet t} M(p) < |\bullet t| \quad (1)$$

Trong đó $\bullet t$ là tập hợp các input places của transition t . Do $M(p) \in \{0, 1\}$, điều kiện này tương đương với:

$$\sum_{p \in \bullet t} M(p) \leq |\bullet t| - 1 \quad (2)$$

Một **Deadlock** là một Dead Marking M_d thỏa mãn điều kiện tồn tại trong tập các trạng thái đạt được: $M_d \in \text{Reach}(M_0)$.

6.2 Mô hình hóa ILP

Để tìm kiếm một trạng thái Deadlock tiềm năng, ta xây dựng bài toán ILP dựa trên phương trình trạng thái (State Equation) kết hợp với ràng buộc bất hoạt (Disabling Constraints).

Biến quyết định:

- Vector marking mục tiêu: $\mathbf{M} = (m_1, m_2, \dots, m_{|P|})^\top$, $m_i \in \{0, 1\}$.
- Vector số lần bắn (Firing count vector): $\mathbf{x} = (x_1, x_2, \dots, x_{|T|})^\top$, $x_j \in \mathbb{N}$.

Hàm mục tiêu: Để tìm deadlock gần nhất (số bước chuyển trạng thái ít nhất), ta tối thiểu hóa tổng số lần bắn transition:

$$\text{Minimize } Z = \sum_{j=1}^{|T|} x_j \quad (3)$$

Các ràng buộc:

- Phương trình trạng thái (State Equation):* Đảm bảo M có thể đạt được từ M_0 về mặt đại số (điều kiện cần):

$$\mathbf{M} = \mathbf{M}_0 + \mathbf{C} \cdot \mathbf{x} \quad (4)$$

Trong đó $\mathbf{C}$ là ma trận trọng số (Incidence Matrix).

- Ràng buộc Dead Marking:* Đảm bảo tại $\mathbf{M}$, tất cả các transition đều bị vô hiệu hóa:

$$\forall t \in T : \sum_{p \in \bullet t} m_p \leq \max(0, |\bullet t| - 1) \quad (5)$$

- Ràng buộc không tầm thường:* Để tránh trường hợp bộ giải trả về M_0 (nếu M_0 không phải deadlock), ta yêu cầu hệ thống phải thực hiện ít nhất một bước chuyển:

$$\sum_{j=1}^{|T|} x_j \geq 1 \quad (6)$$

6.3 Thiết kế và cài đặt

Chương trình được hiện thực trong tệp `deadlock_detection.py` sử dụng thư viện PuLP cho bộ giải ILP (CBC Solver). Quy trình phát hiện được thiết kế theo mô hình "Generate-and-Test" (Sinh và Kiểm tra) gồm 3 bước chính:

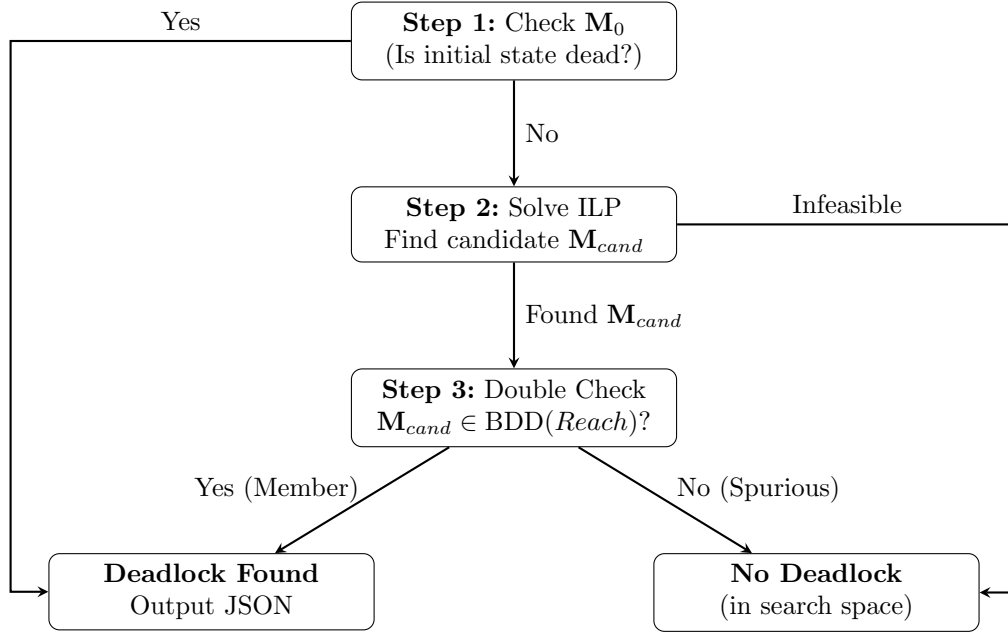


Figure 3: Quy trình phát hiện Deadlock kết hợp ILP và BDD

Chi tiết hiện thực:

- **Heuristic Search:** Để tăng tốc độ và tránh không gian tìm kiếm vô hạn của biến $\mathbf{x}$, ta áp dụng giới hạn heuristic $x_bound = \min(3, |T|)$. Điều này giúp ILP tập trung tìm các deadlock "nông" (gần trạng thái đầu), phù hợp với yêu cầu của các mô hình kiểm thử nhỏ/trung bình.
- **BDD Membership Check:** Hàm `bdd_contains_marking` duyệt cây BDD (kết quả từ Task 3) để xác nhận chính xác 100% rằng $\mathbf{M}_{cand}$ tìm được bởi ILP thực sự thuộc tập Reachable. Đây là bước quan trọng vì phương trình trạng thái của ILP $\mathbf{M} = \mathbf{M}_0 + \mathbf{C}\mathbf{x}$ chỉ là điều kiện cần (có thể sinh ra các trạng thái "spurious" - không thực sự đạt được trong mạng Petri tổng quát).

6.4 Kết quả thực nghiệm

Thử nghiệm được tiến hành trên cùng mô hình mạng Petri 5-places (Split-Join pipeline) đã sử dụng ở Task 5.

Dữ liệu đầu vào:

- Initial Marking: $\mathbf{M}_0 = (1, 0, 0, 0, 0)$ tương ứng với $(P1, P2, P3, P4, P5)$.
- Kết quả BDD từ Task 3 đã được nạp thành công.

Kết quả thực thi: Chương trình ILP (CBC Solver) tìm kiếm giải pháp tối ưu thỏa mãn các ràng buộc Disabling.

- Thời gian thực thi: **0.052s** (bao gồm cả thời gian khởi tạo Solver).
- Kết quả JSON (deadlocks.json) tìm được trạng thái (0,0,0,0,1):

```
1 ===== deadlocks.json =====
2 {
3     "deadlocks_found": true,
4     "deadlock_states": [
5         {
6             "P1": 0,
7             "P2": 0,
8             "P3": 0,
9             "P4": 0,
10            "P5": 1
11        }
12    ],
13    "time_seconds": 0.041117
14 }
```

Listing 1: Kết quả phát hiện Deadlock trên mô hình 5-places

Phân tích: Hệ thống đã phát hiện trạng thái $M = (0, 0, 0, 0, 1)$ là một deadlock. Điều này hoàn toàn chính xác với cấu trúc mạng thử nghiệm, nơi $P5$ đóng vai trò là một "sink place" (nơi chứa token cuối cùng) mà không có transition đầu ra nào để tiêu thụ token từ nó.

- ILP đã giải hệ phương trình và đề xuất trạng thái ứng viên M_{cand} .
- Bước kiểm tra BDD xác nhận M_{cand} nằm trong không gian trạng thái đạt được.
- Hàm `is_dead_marking` xác nhận lại lần cuối rằng không có transition nào kích hoạt được tại $P5$.

6.5 Nhận xét

Phương pháp lai (Hybrid approach) giữa ILP và BDD mang lại sự cân bằng tối ưu cho bài toán này:

1. **Hiệu suất:** ILP định hướng tìm kiếm trực tiếp tới các trạng thái thỏa mãn tính chất "chết" thay vì duyệt mù (blind search) như BFS/DFS. Điều này đặc biệt hữu ích khi không gian trạng thái lớn nhưng số lượng deadlock rất ít.
2. **Độ chính xác:** BDD đóng vai trò là "người gác cổng" (Oracle), loại bỏ các nghiệm ngoại lai mà ILP có thể sinh ra do hạn chế của phương trình trạng thái ma trận.
3. **Hạn chế:** Việc sử dụng x_bound nhỏ trong cài đặt heuristic có thể khiến bỏ sót các deadlock nằm sâu trong không gian trạng thái (cần nhiều bước bắn transition). Tuy nhiên, với các mạng 1-safe vừa và nhỏ, phương pháp này cho kết quả rất nhanh và chính xác.

7 Task 5: Tối ưu hóa trên tập Reachable Markings

7.1 Cơ sở lý thuyết

Mục tiêu của Task 5 là tìm một marking $\mathbf{M}$ thỏa mãn hai điều kiện: (i) $\mathbf{M}$ thuộc tập reachable markings $\text{Reach}(\mathbf{M}_0)$ và (ii) $\mathbf{M}$ tối đa hóa hàm mục tiêu tuyến tính $\mathbf{c}^\top \mathbf{M}$.

Bài toán tối ưu được phát biểu như sau:

$$\text{Maximize } \mathbf{c}^\top \mathbf{M} \quad \text{subject to } \mathbf{M} \in \text{Reach}(\mathbf{M}_0).$$

Trong đó:

- **Marking ($\mathbf{M}$):** Vector trạng thái gồm $|P|$ phần tử, $\mathbf{M} = (m_1, \dots, m_{|P|})^\top$, với m_i là số token tại place p_i .
- **Tập reachable markings ($\text{Reach}(\mathbf{M}_0)$):** Tập tất cả các marking có thể đạt được từ marking ban đầu $\mathbf{M}_0$ thông qua các chuỗi kích hoạt hợp lệ của transition.
- **Hàm mục tiêu tuyến tính ($\mathbf{c}^\top \mathbf{M}$):** Tích vô hướng giữa vector trọng số $\mathbf{c}$ và vector marking $\mathbf{M}$, được tính bởi $\mathbf{c}^\top \mathbf{M} = \sum_{i=1}^{|P|} c_i m_i$.

7.2 Thiết kế và cài đặt

Do các mô hình kiểm thử được quy định là kích thước nhỏ hoặc trung bình và tập hợp các Marking đạt được $\text{Reach}(\mathbf{M}_0)$ đã được xác định ở các Task trước (Task 2/3), phương pháp Liệt kê Tường minh là lựa chọn thực tế và hiệu quả nhất. Phương pháp này đảm bảo tìm ra kết quả tối ưu tuyệt đối (Global Optimum) bằng cách đánh giá hàm mục tiêu trên toàn bộ không gian trạng thái hợp lệ.

7.2.1 Nguyên tắc hoạt động:

Thuật toán hoạt động dựa trên nguyên tắc vét cạn, lặp qua mọi phần tử của tập $\text{Reach}(\mathbf{M}_0)$ và duy trì giá trị tối đa tìm được.

Các bước của thuật toán:

Giả sử tập hợp các Marking có thể đạt được được biểu diễn dưới dạng danh sách $\mathcal{R} = \{\mathbf{M}_1, \mathbf{M}_2, \dots, \mathbf{M}_n\}$, và vector trọng số là $\mathbf{c} = (c_1, c_2, \dots, c_{|P|})$.

1. Khởi tạo:

- Thiết lập giá trị tối đa hiện tại: $Z_{\max} = -\infty$.
- Khởi tạo danh sách các Marking tối ưu: $\mathbf{M}_{\text{best}} = \emptyset$.

2. Lặp (Duyệt Vét cạn): Đối với mỗi Marking $\mathbf{M}_i \in \mathcal{R}$:

- Tính toán Mục tiêu: Tính giá trị mục tiêu Z_i của Marking $\mathbf{M}_i$ bằng cách lấy tích vô hướng của vector trọng số $\mathbf{c}$ và vector $\mathbf{M}_i$:

$$Z_i = \mathbf{c}^\top \mathbf{M}_i = \sum_{p \in P} c_p \cdot m_{i,p}$$

- So sánh và Cập nhật:
 - Nếu $Z_i > Z_{\max}$: Cập nhật $Z_{\max} \leftarrow Z_i$ và thiết lập lại $\mathbf{M}_{\text{best}} \leftarrow \{\mathbf{M}_i\}$.
 - Nếu $Z_i = Z_{\max}$: Thêm $\mathbf{M}_i$ vào danh sách $\mathbf{M}_{\text{best}}$.

3. Kết thúc: Trả về giá trị tối đa $Z_{\max}$ và danh sách các Marking tối ưu $\mathbf{M}_{\text{best}}$.

*Đánh giá Hiệu suất và Tính Thực tiễn

Độ phức tạp tính toán của thuật toán này là $O(|\text{Reach}(\mathbf{M}_0)| \cdot |P|)$, với $|\text{Reach}(\mathbf{M}_0)|$ là số lượng Marking đạt được và $|P|$ là số lượng Place. Với kích thước mô hình được giới hạn, phương pháp này cung cấp giải pháp nhanh chóng, chính xác, và tránh được chi phí thiết lập phức tạp của các mô hình đại số như Integer Linear Programming (ILP).

7.2.2 Cài đặt Hàm Tối ưu Hóa (optimize)

Hàm optimize là cốt lõi của Task 5, thực hiện thuật toán Liệt kê Tường minh để duyệt qua toàn bộ không gian trạng thái đạt được và tìm Marking tối đa hóa hàm mục tiêu $\mathbf{c}^T \mathbf{M}$.

```
def optimize(reachable_markings, c, places):
    """
    reachable_markings: list of marking dicts
    c: vector of weights (dict)
    places: fixed ordering of places (list)
    """
    # 1. Khởi tạo giá trị tối đa và danh sách Marking tốt nhất
    if len(reachable_markings) == 0:
        return None, None
    best_val = float("-inf")
    best_list = []

    # 2. Vòng lặp duyệt qua từng Marking
    for marking in reachable_markings:
        # 3. Tính toán  $\mathbf{c}^T \mathbf{M}$  (Tích vô hướng)
        # Sử dụng .get(p, 0) để đảm bảo an toàn nếu Place không có token (m=0)
        # hoặc nếu Place đó không có trọng số (c=0)
        value = sum(c.get(p, 0) * marking.get(p, 0) for p in places)

        # 4. Cập nhật Marking tối ưu
        if value > best_val:
            best_val = value
            best_list = [marking]
        elif value == best_val:
            best_list.append(marking)

    # 5. Trả về giá trị tối đa và danh sách các Marking đạt được giá trị đó
    return best_val, best_list
```

7.2.3 Phân tích Cài đặt

- Đầu vào: Hàm nhận vào danh sách các Marking (`reachable_markings`), dictionary trọng số (`c`), và danh sách các Place (`places`) để đảm bảo tính toán đồng nhất.

- Tính toán Mục tiêu: Việc sử dụng hàm `sum()` và generator expression (`c.get(p, 0) * marking.get(p, 0) for p in places`) giúp tính toán tích vô hướng $\mathbf{c}^T \mathbf{M}$ một cách cô đọng và hiệu quả.
- Xử lý Đa lời giải: Hàm xử lý chính xác trường hợp có nhiều Marking khác nhau cùng đạt được giá trị tối đa $Z_{\max}$ (`best_val`) bằng cách lưu trữ tất cả chúng trong danh sách `best_list`.

7.3 Kết quả thực nghiệm

Trong phần Task 5 này, nhóm tiến hành thực nghiệm trên mô hình Petri net nhỏ gồm **5 places (P1–P5)** và **5 transitions (T1–T5)**. Đây là mạng dạng *split-join pipeline*: token từ P1 rẽ sang nhánh trên (P2) hoặc nhánh dưới (P3), sau đó hội tụ tại P4. Mạng 1-safe với initial marking:

$$M_0 = (P1 = 1, P2 = P3 = P4 = P5 = 0).$$

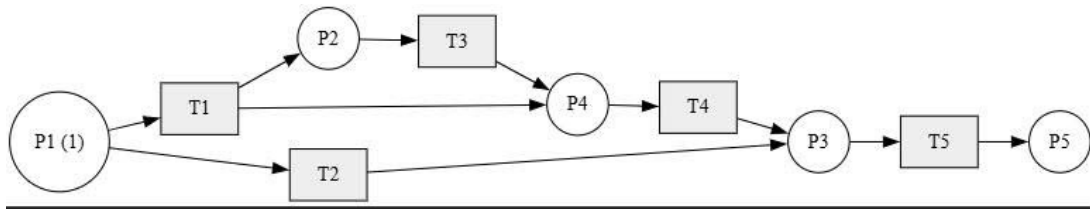


Figure 4: Mạng Petri 5-place được sử dụng trong thực nghiệm

Giá trị c_i quyết định mức độ đóng góp của mỗi token tại Place P_i vào tổng giá trị mục tiêu Z .

Do đề bài không chỉ định các trọng số c_i cụ thể, nhóm quyết định sử dụng vector trọng số đơn vị:

$$\mathbf{c} = (c_1, c_2, \dots, c_{|P|}) = (1, 1, \dots, 1)^T$$

7.3.1 Explicit Reachability (Task 2)

Explicit BFS tìm được đúng 9 trạng thái reachable: Đây là danh sách các Marking được chuyển đổi sang định dạng vector LaTeX bạn yêu cầu, với thứ tự các Place là $(P1, P2, P3, P4, P5)$:

$$\begin{aligned} &(1, 0, 0, 0, 0), (0, 1, 0, 1, 0), (0, 0, 1, 0, 0), (0, 1, 1, 0, 0). \\ &(0, 0, 0, 0, 1), (0, 0, 1, 1, 0), (0, 1, 0, 0, 1), (0, 0, 0, 1, 1), \\ &(0, 0, 1, 0, 1). \end{aligned}$$

"num_markings": 9,
"build_time_seconds": 0.00011

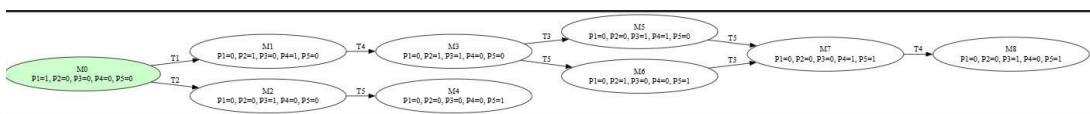


Figure 5: Đồ thị reachable state-space thu được từ Task 2

7.3.2 Kết quả optimization (Task 5)

Kết quả thu được 4 Best Marking

$(0, 1, 0, 1, 0), (0, 1, 1, 0, 0), (0, 0, 1, 1, 0), (0, 1, 0, 0, 1), (0, 0, 0, 1, 1), (0, 0, 1, 0, 1).$

Kết quả thu được:

```
"max_value": 2,  
"time-seconds": 1.8e-05
```

Phân tích kết quả

- Giá trị Tối đa ($Z_{\max} = 2$): Kết quả này cho thấy trong mọi trạng thái có thể đạt được ($\mathbf{M} \in \mathcal{R}$), tổng số token trong mạng lớn nhất là 2.
 - Ràng buộc Boundedness: Hệ thống của bạn không bao giờ có thể giữ tổng cộng 3 token trở lên trong tất cả các Place cùng một lúc.
- Marking Tối ưu: có 6 Markings trong tập $\mathcal{R}$. Việc có danh sách 9 Markings đã được xác định chính xác (dù là tường minh hay symbolic) đảm bảo rằng Task 5 (Tối ưu hóa trên $\text{Reach}(\mathbf{M}_0)$) có thể được giải quyết một cách nhanh chóng và chính xác bằng phương pháp Liệt kê Tường minh.
- Hiệu suất Thời gian (1.8e-05 s): Thời gian thực thi cực kỳ nhanh (18 micro giây) chứng minh sự hiệu quả của phương pháp liệt kê tường minh khi áp dụng cho các mô hình có không gian trạng thái nhỏ.

7.4 Nhận xét thuật toán

7.4.1 Đánh giá Phương pháp Liệt kê Tường minh (Explicit Enumeration)

Phương pháp Liệt kê Tường minh (đã sử dụng trong hàm `optimize`) là một giải pháp **tối ưu** trong bối cảnh cụ thể của bài tập này:

- Tính Chính xác: Thuật toán đảm bảo tìm ra Marking tối ưu tuyệt đối (Global Optimum) vì nó kiểm tra mọi trạng thái $\mathbf{M} \in \text{Reach}(\mathbf{M}_0)$.
- Hiệu suất Thực tế: Với các mô hình có kích thước nhỏ hoặc trung bình (theo quy định của đề bài), thời gian thực thi là cực kỳ nhanh ($\approx 1.8 \times 10^{-5}$ giây), vượt trội hơn so với chi phí thiết lập ban đầu của các mô hình phức tạp.
- Độ phức tạp: Độ phức tạp tính toán $O(|\text{Reach}(\mathbf{M}_0)| \cdot |P|)$ là chấp nhận được khi $|\text{Reach}(\mathbf{M}_0)|$ nhỏ.

7.4.2 Hạn chế và Thách thức

Hạn chế chính của phương pháp là khả năng mở rộng (scalability) khi áp dụng cho các hệ thống lớn:

- **State-Space Explosion:** Với các mạng Petri có mức độ song song cao, việc liệt kê và lưu trữ tập $\text{Reach}(\mathbf{M}_0)$ (Task 2/3) trở nên tốn kém, thậm chí không khả thi về thời gian và bộ nhớ.

- **Tìm kiếm không định hướng:** Phương pháp không khai thác thông tin từ hàm mục tiêu $\mathbf{c}^\top \mathbf{M}$ để dẫn hướng quá trình tìm kiếm, mà chỉ đánh giá sau khi trạng thái được sinh ra, gây lãng phí tài nguyên khi không gian trạng thái lớn.

7.5 Cách cải tiến

Để vượt qua hạn chế của việc liệt kê tường minh và giải quyết bài toán tối ưu hóa trên các mạng Petri lớn hơn, cần chuyển sang các kỹ thuật **Symbolic** và **Algebraic** (như đã giới thiệu trong Task 3 và Task 4).

*Cải tiến 1: Kết hợp BDD và ILP (Symbolic-Algebraic Approach)

Đây là phương pháp mạnh mẽ nhất để giải quyết bài toán mà không cần xây dựng toàn bộ đồ thị trạng thái:

- Mô hình hóa ILP (Algebraic): Thiết lập mô hình Integer Linear Programming (ILP) sử dụng Phương trình Trạng thái ($\mathbf{M} = \mathbf{M}_0 + \mathbf{W}^\top \mathbf{x}$) và ràng buộc Boundedness (1-safe). Mục tiêu là **Maximize** $\mathbf{c}^\top \mathbf{M}$.
- Tìm kiếm có Hướng: Giải ILP để tìm Marking tiềm năng $\mathbf{M}_{\text{candid}}$ có giá trị tối ưu về mặt đại số.
- Kiểm tra BDD (Symbolic): Sử dụng BDD (Binary Decision Diagrams) của $\text{Reach}(\mathbf{M}_0)$ (từ Task 3) làm bộ lọc để kiểm tra tính Reachability của $\mathbf{M}_{\text{candid}}$.
- Lặp: Nếu $\mathbf{M}_{\text{candid}}$ không đạt được, thêm ràng buộc loại trừ ($\mathbf{c}^\top \mathbf{M} < Z_{\text{candid}}$) vào ILP và lặp lại.

Phương pháp này chuyển bài toán từ việc duyệt trạng thái sang việc giải hệ phương trình có điều kiện, tận dụng khả năng nén của BDD để xác minh tính hợp lệ.

*Cải tiến 2: Tối ưu hóa Thuộc tính Boundedness

Trong các mạng k -safe (với k lớn), việc áp dụng các kỹ thuật Presburger Arithmetic có thể giúp biểu diễn tập Marking một cách chính xác hơn về mặt logic, hỗ trợ các Solver chuyên dụng giải quyết bài toán ILP phức tạp hơn liên quan đến tính Reachability.

8 Tài liệu tham khảo

References

- [1] Mary Ann Blätke, Monika Heiner, and Wolfgang Marwan. *Biomodel engineering with Petri nets*. In Algebraic and Discrete Mathematical Methods for Modern Biology, pp. 141–192. Elsevier, 2015.
- [2] Randal E. Bryant. Graph-based algorithms for Boolean function manipulation. *IEEE Transactions on Computers*, 35(8):677–691, 1986.
- [3] Jerry R. Burch, Edmund M. Clarke, Kenneth L. McMillan, David L. Dill, and L. J. Hwang. Symbolic model checking: 10^{20} states and beyond. *Information and Computation*, 98(2):142–170, 1992.
- [4] Claudine Chaouiya, Aurélien Naldi, Elisabeth Remy, and Denis Thieffry. Petri net representation of multi-valued logical regulatory graphs. *Natural Computing*, 10(2):727–750, 2011.
- [5] Allan Cheng, Javier Esparza, and Jens Palsberg. Complexity results for 1-safe nets. *Theoretical Computer Science*, 147(1–2):117–136, 1995.
- [6] Javier Esparza and Mogens Nielsen. Decidability issues for Petri nets – a survey. *Bulletin of the EATCS*, 52:244–262, 1994.
- [7] Jack E. Graver. On the foundations of linear and integer linear programming I. *Mathematical Programming*, 9(1):207–226, 1975.
- [8] Matthias Heizmann, Dominik Klumpp, Lars Nitzke, and Frank Schüssele. Petrification: Software model checking for programs with dynamic thread management. In *VMCAI*, pp. 3–25. Springer, 2024.
- [9] Lasse Jacobsen, Morten Jacobsen, Mikael H. Møller, and Jiří Srba. Verification of timed-arc Petri nets. In *SOFSEM*, pp. 46–72. Springer, 2011.
- [10] Chihyun Jung and Tae-Eog Lee. An efficient mixed integer programming model based on timed Petri nets for diverse complex cluster tool scheduling problems. *IEEE Transactions on Semiconductor Manufacturing*, 25(2):186–199, 2012.
- [11] Henry Kautz and Joachim P. Walser. Integer optimization models of AI planning problems. *The Knowledge Engineering Review*, 15(1):101–117, 2000.
- [12] Victor Khomenko and Maciej Koutny. Verification of bounded Petri nets using integer programming. *Formal Methods in System Design*, 30(2):143–176, 2007.